Some 64 Instructions

PUSH	push value onto the stack
POP	pop the top of the stack
CALL	jump to function code
RET,RETN,RETF	return from a function call
MOV	move values between registers
JMP	jump to another address in the code
CMP	compare 2 values by subtracting them.
TEST	do the bitwise "and" of 2 values.
JE	jump if equal
JNE	jump if not equal
JZ	jump if zero
JNZ	jump if not zero
NOP	no operation, i.e., do nothing just skip over it.

Jumps refer to the values used in the preceding CMP or TEST.
More information is available at: `http://www.cs.virginia.edu/~evans/cs216/guides/x86.html`

Example Program in C

```
#include <stdio.h>

int incFunction( int x ){
   return x+1;
}

int main() {
   int i;
   for (i=0;i<10;i++) {
     printf("%d \n",incFunction(i));
   }
   return 0;
}
```

The x64 Assembly For This Program

```
_incFunction
    push    rbp                 // store the old frame pointer on the stack
    mov     rbp, rsp            // set the base of the new stack frame
    mov     [rbp+var_4], edi    // move function argument to var_4 on stack.
    mov     eax, [rbp+var_4]    // move this into eax (the return result register)
    add     eax, 1              // add 1 to eax
    pop     rbp                 // reset the old stack frame base pointer
    retn                        // pop address of stack and return to it
```

```
public _main
    push    rbp                 // store the old frame pointer on the stack
    mov     rbp, rsp            // set the base of the new stack frame
    sub     rsp, 10h            // Move the top of the stack up 10h
    mov     [rbp+var_4], 0.     // set var_4 (on stack) to zero
    mov     [rbp+var_8], 0      // set var_4 (on stack) to zero, this is i
loc_100003F56:
    cmp     [rbp+var_8], 0Ah    // compare var_8 (i) with Ah (10).
    jge     loc_100003F86.      // if i >= 10 jump to loc_100003F86
    mov     edi, [rbp+var_8]    // load var_8 (i) into edi (first argument of function)
    call    _incFunction        // execute function  _incFunction, result in eax
    lea     rdi, aD  "%d \n"    // load "%d \n" into edi (first argument)
    mov     esi, eax            // move eax (results of incFunction) into esi, 2nd arg.
    mov     al, 0               // zero first byte of rax
    call    _printf             // _printf("%d \n",incFunction(i))
    mov     eax, [rbp+var_8]    // load i into eax
    add     eax, 1              // calculate i +1
    mov     [rbp+var_8], eax    // put i= i + i
    jmp     loc_100003F56       // loop
loc_100003F86:
    xor     eax, eax            // zero eax (4 bytes of rax = one int).
    add     rsp, 10h            // reset the pointer to the top of the stack
    pop     rbp                 // reset the old stack frame base pointer
    retn                        // pop address of stack and return to it
```

Some ARM Instructions

LDR	load memory into a register
LDP	load a pair of values from memory into a register
STR	storea register to memory
STP	store a pair of values to memory
MOV	move values between registers
ADD	add two values
ADDS	add two values and set comparison flags
SUB	subtract two values
SUBS	subtract two values and set comparison flags
CMP	compare 2 values by subtracting them.
B	jump to another address in the code
BZ, BEQ, ...	conditional branch based on comparison flags
BL	jump to function code
CSET	set registier to 1 based on comparison flags
TBNX	branch is bit of register is not zero
RET	return from a function call

The ARM AArch64 Assembly For This Program

```
_incFunction
    sub      sp,sp,#0x10        // move up the top of the stack
    str      w0,[sp, #local_4]  // store first arg (w0) on stack.
    ldr      w8,[sp, #local_4]  // load this value into register w8.
    add      w0,w8,#0x1         // w0 = w8 +1
    add      sp,sp,#0x10        // reset the stack
    ret                         // return with w0 as the result.

_main
entry
    sub      sp,sp,#0x20              // move up the top of the stack
    stp      x29,x30,[sp, #local_10] // store link register and frame pointer on stack
    add      x29,sp,#0x10.            // set frame pointer (x29) for this function
    stur     wzr,[x29, #local_14]     // write zero to local_14 (idk)
    str      wzr,[sp, #local_18]      // write zero to local_18 (this is i)
    b        LAB_100003f34            // completely unnecessary branch
LAB_100003f34:
    ldr      w8,[sp, #local_18]       // load local_18 (i) into register w8.
    subs     w8,w8,#0xa               // subtract 10 (a in hex) from w8
    cset     w8,ge                    // write 1 to w8 if w8 > 10
    tbnz     w8,#0x0,LAB_100003f7c    // w8 is not zero (i.e., i>10) branch.
    b        LAB_100003f48            // completely unnecessary branch
LAB_100003f48
    ldr      w0,[sp, #local_18]       // load local_18 (i) into register w0
    bl       _incFunction             // call _incFunction with argument w0
    mov      x9,sp                    // copy stack pointer to x9
    mov      x8,x0                    // move result of _incFunction to x8
    str      x8,[x9]                  // store x8  at x9 (top of stack)
    adrp     x0,0x100003000           // load current memory address
    add      x0=>s_%d_100003f98,x0,#0xf98   // set x0   -> "%d \n"
    bl       __stubs::_printf         // branch to printf
    b        LAB_100003f6c
LAB_100003f6c
    ldr        w8,[sp, #local_18]      \\
    add        w8,w8,#0x1              || local_18 (i) = local_18 (i) +1
    str        w8,[sp, #local_18]      //
    b          LAB_100003f34
LAB_100003f7c
    mov        w0,#0x0                 // set w0 (the function result) to equal 0
    ldp        x29=>local_10,x30,[sp, #0x10].   // restore x29 (fp) and x30 (lr)
    add        sp,sp,#0x20.            // rest sp (the top of the stack)
    ret
```